DATA ENGINEERING CHALLENGE

Data Engineering Challenge

Wearable Health Data Pipeline

Duration **7 days**

SECTION 1

Context

Meridian Health Analytics runs a research program that collects continuous wearable data from enrolled study participants - activity, heart rate, and sleep - alongside daily self-reported wellness surveys. The raw data lands from the field largely untouched: exported from devices and companion apps and dropped into the repository as-is.

You are joining as the data engineer who owns the platform that turns those raw exports into data the organization can trust. The raw study data sits in S3 at `s3://de-tech-assessment-396587179375-us-east-1-an/data/` - a study covering **50 participants over a 28-day window**, arriving from several sources in a mix of formats and grains. A data dictionary ships alongside this brief; everything else - the shape of the data, its quirks, and how to model it - is yours to discover.

A DESIGN DECISION YOU MUST MAKE

Two teams consume this data and they do not want it the same way. The clinical analytics team needs **daily freshness**. The biostatistics team works in **weekly batches** and expects a **different schema**. How you serve both consumers from one platform is a decision you will need to make and justify.

SECTION 2

The mandate

Deliver a pipeline that takes the raw study exports and produces analysis-ready tables the two consumer teams can rely on. We are giving you **outcomes, not a blueprint** - no prescribed layers, schemas, storage engine, or scaffolding. The architecture is yours to design and defend.

1 **Make the data trustworthy.**
This is real-world exported data and it is not clean. Profile it, find what is wrong, and decide what "correct" means for each source - then enforce it. Nothing about the data's problems is disclosed to you; discovering and handling them is the heart of the exercise.

2 **Model it for its consumers.**
Design tables that serve daily clinical analytics and weekly biostatistics from a single source of truth, including the analytical layer needed to compare participant cohorts by chronotype (sleep and activity).

3 **Engineer it to production standards.**
Correctness, reproducibility, and idempotency are expected: re-running the pipeline must not duplicate or corrupt data. Consider scale, cost, testing, and how you would operate this beyond a one-off run.

4 **Justify your choices.**

Storage format, modeling approach, data-quality strategy, and orchestration are all decisions we want to see reasoned through - including the trade-offs you rejected.

Use whatever stack you would actually reach for. There is no required technology.

SECTION 3

What we evaluate

We assess the delivery across these dimensions:

Data quality

How thoroughly you find, correct, and guard against the problems in the raw data - and how you make that verifiable.

Modeling

Whether the layers and schemas are correct, join-ready, and well-fitted to both consumers.

Engineering

Idempotency, reproducibility, testing, structure, and readiness to run and operate.

Judgment

The quality of your design reasoning and the trade-offs you chose to make - or not.

SECTION 4

Deliverables

- [ ] **Git repository**
  Zipped or hosted, containing your pipeline code and a README that explains how to run it end to end.

- [ ] **Architecture diagram**
  A simple diagram of your pipeline showing the main layers, how data flows between them, and where data quality is handled. The solution should be containerized so that it can be run locally, while the architecture should also demonstrate how the same solution could be deployed and run using cloud services. Clearly indicate the local components and their corresponding cloud services. A sketch or a few boxes is sufficient. Include a short README with any key architectural decisions or assumptions worth calling out.

- [ ] **Analytical query output**
  A query result against your analytical layer demonstrating at least three metrics, including a chronotype cohort comparison.

- [ ] **AI usage disclosure**
  A short section in your README outlining any use of AI-assisted tools (e.g. code generation, debugging, design suggestions).

A follow-up technical discussion will explore your design decisions and how you would evolve the pipeline as the study scales.